DeliverMe Socket.IO Documentation

This document maps the active Socket.IO connections in the DeliverMe app. It was reviewed from the project files while excluding `node_modules` and the draft folder (`backend/Drafts`).

Scope

Active socket files reviewed:

- Backend server bootstrap: `backend/server.mjs`
- Backend socket gateway: `backend/src/socket/SocketIndex.mjs`
- Backend client events: `backend/src/socket/client.socket.mjs`
- Backend driver events: `backend/src/socket/driver.socket.mjs`
- Backend ride worker broadcasts: `backend/src/workers/rideMatching.worker.mjs`
- Frontend socket singleton: `frontend/src/services/SocketManager.js`
- Frontend client socket helpers: `frontend/src/services/ClientSocket.js`
- Frontend driver socket helpers: `frontend/src/services/DriverSocket.js`
- Frontend background location sender: `frontend/src/services/backgroundLocationService.js`
- Frontend driver socket UI listeners/emitters: `frontend/src/screens/DriverHomeScreen.js`
- Frontend client socket startup: `frontend/src/screens/ClientHomeScreen.js`

High-Level Architecture

The backend creates one HTTP server and attaches Socket.IO to it in `backend/server.mjs:22`. The same `io` instance is stored at `app.locals.io` and initialized through `initSocket(io)` in `backend/server.mjs:34`.

The frontend creates a singleton Socket.IO client in `frontend/src/services/SocketManager.js:26`. It connects to:

```
const SOCKET_URL = "http://10.114.106.200:5000";
```

The frontend sends the JWT from `AsyncStorage` as the Socket.IO handshake auth token:

```
io(SOCKET_URL, {
  transports: ["websocket"],
  auth: { token },
  reconnection: true,
  reconnectionAttempts: Infinity,
  reconnectionDelay: 1000,
  upgrade: false,
});
```

The backend validates that token in `backend/src/socket/SocketIndex.mjs:72`. After verification, the backend attaches this user object to the socket:

```
{
  id: decoded.id,
  name: decoded.name,
  mobile: decoded.mobile,
  role: decoded.role,
  exp: decoded.exp
}
```

Then the socket joins one role-scoped room:

- Driver room: `driver:{driverId}` in `backend/src/socket/SocketIndex.mjs:103`
- Client room: `client:{clientId}` in `backend/src/socket/SocketIndex.mjs:113`

Current active backend emits target driver rooms. Client rooms exist but no active server-to-client app event currently emits to them.

Connection And Access Flow

1. User signs in through REST and receives a JWT.
2. Frontend stores JWT in `AsyncStorage` as `userToken`.
3. `initSocket()` reads `userToken` and `userType` in `frontend/src/services/SocketManager.js:33`.
4. Socket.IO connects with `{ auth: { token } }`.
5. Backend verifies the JWT in `backend/src/socket/SocketIndex.mjs:72`.
6. Backend joins either `driver:{id}` or `client:{id}`.
7. Backend registers role-specific events:

- Driver events through `registerDriverSocket(io, socket)` in `backend/src/socket/SocketIndex.mjs:109`
- Client events through `registerClientSocket(io, socket)` in `backend/src/socket/SocketIndex.mjs:117`

1. Most custom events validate token expiry again inside their event handler.

Event Catalog

Lifecycle Events

Event	Direction	Emitted By	Received By	Payload	Notes
`connect`	Socket.IO internal	Socket.IO client/server	`frontend/src/services/SocketManager.js:47`	Built-in socket connection data	Frontend reacts by emitting `driverOnline` for drivers or `clientOnline` for clients.
`connection`	Socket.IO internal	Socket.IO server	`backend/src/socket/SocketIndex.mjs:99`	Server `socket` object	Backend joins role room and registers event handlers.
`disconnect`	Socket.IO internal	Either side	Backend: `backend/src/socket/SocketIndex.mjs:119`; frontend: `frontend/src/services/SocketManager.js:64`	`reason: string`	Used for logging only.
`connect_error`	Socket.IO internal	Socket.IO client	`frontend/src/services/SocketManager.js:70`	`Error` with `message`	If message is `Unauthorized`, frontend removes `userToken` and disconnects.

Client Events

Event	Direction	Emitted By	Received By	Payload Type	Ack Type	
`clientOnline`	Frontend -> Backend	`frontend/src/services/SocketManager.js:57`; `frontend/src/services/ClientSocket.js:42`; `frontend/src/services/ClientSocket.js:48`	`backend/src/socket/client.socket.mjs:23`	Optional object: `{ message?: string }` or no payload	`{ ok: true }` or `{ ok: false, reason: "TOKEN_EXPIRED" \`	"SERVER_ERROR" }`
`client:getId`	Frontend -> Backend	`frontend/src/services/ClientSocket.js:15`	`backend/src/socket/client.socket.mjs:37`	`null`	`{ ok: true, clientId: string }` or `{ ok: false, reason: "TOKEN_EXPIRED" \`	"SERVER_ERROR" }`

Example `clientOnline` payload:

```
{
  "message": "Client is online"
}
```

Example `client:getId` call:

```
socket.emit("client:getId", null, (ack) => {
  // ack = { ok: true, clientId: "65f1..." }
});
```

Driver Events

Event	Direction	Emitted By	Received By	Payload Type	Ack Type			
`driverOnline`	Frontend -> Backend	`frontend/src/services/SocketManager.js:51` ; `frontend/src/services/DriverSocket.js:59`	`backend/src/socket/driver.socket.mjs:68`	Optional coordinates object or no payload	`{ ok: true }` or `{ ok: false, reason: "TOKEN_EXPIRED" \`	"SERVER_ERROR" }`		
`driverLocation`	Frontend -> Backend	`frontend/src/services/DriverSocket.js:101` ; buffered resend at `frontend/src/services/DriverSocket.js:203`	`backend/src/socket/driver.socket.mjs:94`	Coordinates object	`{ ok: true, reason: "SUCCESS" }` , `{ ok: false, reason: "INVALID_COORDS" \`	"REDIS_ERROR" \	"TOKEN_EXPIRED" \	"SERVER_ERROR" }`
`driverHeartbeat`	Frontend -> Backend	`frontend/src/services/DriverSocket.js:101`	`backend/src/socket/driver.socket.mjs:126`	Empty object `{}`	`{ ok: true }` or `{ ok: false, reason: "TOKEN_EXPIRED" \`	"SERVER_ERROR" }`		
`driver:register`	Frontend -> Backend	Restore online: `frontend/src/screens/DriverHomeScreen.js:360` ; toggle online: `frontend/src/screens/DriverHomeScreen.js:474`	`backend/src/socket/driver.socket.mjs:151`	Object currently sent as `{ driverId: string, isAvailable: true }`	`{ ok: true }` or `{ ok: false, reason: "TOKEN_EXPIRED" \`	"SERVER_ERROR" }`		
`driver:availability`	Frontend -> Backend	Logout offline: `frontend/src/screens/DriverHomeScreen.js:400` ; toggle online: `frontend/src/screens/DriverHomeScreen.js:481` ; toggle offline: `frontend/src/screens/DriverHomeScreen.js:484`	`backend/src/socket/driver.socket.mjs:169`	`{ isAvailable: boolean }`	`{ ok: true }` or `{ ok: false, reason: "TOKEN_EXPIRED" \`	"SERVER_ERROR" }`		

Example driver coordinates:

```
{
  "latitude": 30.0444,
  "longitude": 31.2357,
  "accuracy": 15
}
```

Backend validation for `driverLocation` requires:

```
type DriverCoords = {
  latitude: number;  // -90 to 90
  longitude: number; // -180 to 180
  accuracy?: number;
};
```

Example `driver:register` payload:

```
{
  "driverId": "65f1a7c8e4b0a12345678901",
  "isAvailable": true
}
```

Important: the backend currently ignores the `driver:register` payload and uses `socket.user.id` from the JWT instead.

Example `driver:availability` payload:

```
{
  "isAvailable": false
}
```

Backend-To-Frontend Events

Event	Direction	Emitted By	Received By	Payload Type	Notes
`ride_request`	Backend worker -> Driver frontend	`backend/src/workers/rideMatching.worker.mjs:141`	`frontend/src/screens/DriverHomeScreen.js:242`	Single ride object	Sent to room `driver:{driverId}` for every nearby alive driver.

Event	Direction	Emitted By	Received By	Payload Type	Notes
`nearby-rides`	Backend -> Driver frontend	Redis publish at `backend/src/workers/rideMatching.worker.mjs:173`; Redis subscribe and socket emit at `backend/src/socket/SocketIndex.mjs:51` and `backend/src/socket/SocketIndex.mjs:58`	`frontend/src/screens/DriverHomeScreen.js:241`	Array of ride objects	Sent when driver availability worker finds pending rides near a newly online driver.

Ride client identity fields:

- `clientName` and `clientMobile` are added by the REST ride request backend from the verified client JWT.
- They are stored in MongoDB and Redis with the ride.
- `ride_request` and `nearby-rides` include those fields because both events send the stored ride object.
- `DriverHomeScreen` displays `clientName` only. It does not display `clientMobile`.

Example `ride_request` payload:

```
{
  "_id": "65f1b123e4b0a12345678902",
  "clientId": "65f1a7c8e4b0a12345678901",
  "clientName": "Client User",
  "clientMobile": "01012345678",
  "pickup": {
    "type": "Point",
    "coordinates": [31.2357, 30.0444]
  },
  "destination": {
    "type": "Point",
    "coordinates": [31.2500, 30.0600]
  },
  "pickupAddress": "Tahrir Square Cairo",
  "destinationAddress": "Nasr City Cairo",
  "fare": 120,
  "status": "pending",
  "assignedDriverId": null,
  "routeDistance": 8500,
  "createdAt": "2026-05-01T09:00:00.000Z",
  "expiresAt": "2026-05-01T09:02:00.000Z"
}
```

Example `nearby-rides` payload:

```
[
  {
    "_id": "65f1b123e4b0a12345678902",
    "clientId": "65f1a7c8e4b0a12345678901",
    "clientName": "Client User",
    "clientMobile": "01012345678",
    "pickup": {
      "type": "Point",
      "coordinates": [31.2357, 30.0444]
    },
    "destination": {
      "type": "Point",
      "coordinates": [31.2500, 30.0600]
    },
    "pickupAddress": "Tahrir Square Cairo",
    "destinationAddress": "Nasr City Cairo",
    "fare": 120,
    "status": "pending",
    "routeDistance": 8.5,
    "expiresAt": "2026-05-01T09:02:00.000Z"
  }
]
```

Ride object shape used by the frontend:

```
type Ride = {
  _id?: string | object;
  rideId?: string;
  id?: string;
  clientId?: string | object;
  clientName?: string;
  clientMobile?: string;
  pickup?: {
    type?: "Point";
    coordinates?: [number, number]; // [longitude, latitude]
    latitude?: number;
    longitude?: number;
  };
  destination?: {
    type?: "Point";
    coordinates?: [number, number]; // [longitude, latitude]
    latitude?: number;
    longitude?: number;
  };
  pickupAddress?: string;
  destinationAddress?: string;
  fare?: number;
  status?: "pending" | "expired" | string;
  assignedDriverId?: string | null;
  routeDistance?: number;
  createdAt?: string | Date;
  expiresAt?: string | Date;
};
```

`DriverHomeScreen` deduplicates ride requests using `_id`, `rideId`, `id`, nested `ride._id`, or a fallback key based on addresses, fare, coordinates, and `expiresAt`.

Redis Pub/Sub Used By Socket Bridge

These are not frontend socket events, but they feed socket events:

Redis Channel	Published By	Subscribed By	Socket Event Produced	Payload
`driver:rides`	`backend/src/workers/rideMatching.worker.mjs:173`	`backend/src/socket/SocketIndex.mjs:51`	`nearby-rides`	`{ driverId: string, location: DriverCoords, rides: Ride[] }`
`driver:offline`	`backend/src/workers/rideMatching.worker.mjs:182`	No active subscriber found	None	`{ driverId: string }`

Example Redis `driver:rides` message:

```
{
  "driverId": "65f1a7c8e4b0a12345678901",
  "location": {
    "latitude": 30.0444,
    "longitude": 31.2357
  },
  "rides": []
}
```

Storage And Presence Keys

Driver socket events update Redis presence and geo state:

- `drivers:geo` : Redis GEO set containing driver IDs by longitude/latitude.
- `driver:{driverId}:online` : string key with TTL.
- `driver:{driverId}:socket` : string key containing the socket ID with TTL.
- `driver:{driverId}:alive` : string key with TTL.

Ride matching uses:

- `rides:geo` : Redis GEO set containing ride IDs by pickup longitude/latitude.
- `ride:{rideId}` : JSON string cache for pending ride data.

Frontend Socket Usage By Screen

Client Home

`frontend/src/screens/ClientHomeScreen.js:149` calls `initSocket()` when the screen mounts.

Then it calls:

- `confirmClientOnline()` at `frontend/src/screens/ClientHomeScreen.js:150`
- `getClientSocketID()` at `frontend/src/screens/ClientHomeScreen.js:153`

The actual ride request is not a socket event. It is a REST request through `requestRide()` in `frontend/src/services/api.js`, which hits `POST /api/auth/client/request-ride`. The backend stores the ride, adds it to Redis, and queues ride matching. The worker later emits `ride_request` to drivers.

Driver Home

`frontend/src/screens/DriverHomeScreen.js` listens for incoming rides:

- Removes old listeners at lines `238` and `239`.
- Receives `nearby-rides` at line `241`.
- Receives `ride_request` at line `242`.

When restoring online state, it:

- Calls `initSocket()` at line `342`.
- Calls `emitDriverOnline(coords)` at line `356`.
- Emits `driver:register` at line `360`.

When toggling online, it:

- Calls `initSocket()` at line `454`.
- Emits `driver:register` at line `474`.
- Calls `emitDriverOnline(coords)` at line `480`.
- Emits `driver:availability` with `{ isAvailable: true }` at line `481`.

When toggling offline or logging out, it emits `driver:availability` with `{ isAvailable: false }` at lines `400` and `484`.

Background Location

`frontend/src/services/backgroundLocationService.js` calls `emitLocation()` from `frontend/src/services/DriverSocket.js`:

- No location data available: line `44`, sends heartbeat.
- First location update: line `59`, sends `driverLocation`.
- Distance threshold reached: line `80`, sends `driverLocation`.
- Not enough movement: line `88`, sends heartbeat.

`emitLocation()` decides the event name:

```
coords === null ? "driverHeartbeat" : "driverLocation"
```

That decision happens in `frontend/src/services/DriverSocket.js:101`.

Currently Commented Or Unused Socket Paths

- `rideExpired` emits are commented out in `backend/src/workers/rideExpiration.worker.mjs:55` and `backend/src/workers/rideExpiration.worker.mjs:58`.
- `driver:offline` is published to Redis in `backend/src/workers/rideMatching.worker.mjs:182`, but no active subscriber was found in the reviewed app files.
- `client:{clientId}` rooms are joined in `backend/src/socket/SocketIndex.mjs:113`, but no active backend socket event currently emits to client rooms.

Implementation Notes From Review

- `SocketManager` emits `driverOnline` or `clientOnline` immediately on `connect`. Driver and client helper files may emit those same events again with payload/ack callbacks. This is intentional in current code, but it means the backend can receive duplicate online events.
- `driver:register` sends `driverId` and `isAvailable` from the frontend, but the backend ignores that payload and trusts the JWT user ID.
- `driverLocation` requires valid latitude/longitude. `driverHeartbeat` should be used when the driver has not moved enough or no location array is available.
- `ride_request` and `nearby-rides` both deliver ride objects to the same driver screen. `ride_request` appends one ride, while `nearby-rides` replaces the current list with a deduped array.
- The frontend socket URL is hardcoded in `frontend/src/services/SocketManager.js`, while the backend port/host are environment-driven in `backend/server.mjs`.